

Algoritma Analizi ve Tasarımı

İleri Düzey Final Sınavı Hazırlık Notları

Bu doküman, final sınavında sorulacak kritik konseptleri sadece teorik olarak değil; sürdürülebilir mimari, performans optimizasyonu ve temiz kod (Clean Code) standartları çerçevesinde ele almaktadır.

1. Sözde Kod (Pseudocode) İçin Asimptotik Analiz

Bir algoritmanın donanımdan bağımsız olarak ne kadar sürede çalışacağını belirlemek için girdi boyutunun (n) sonsuza gitmesi durumu incelenir. İç içe döngüler iteratif analizle, kendi kendini çağıran fonksiyonlar ise rekürsif analizle hesaplanır.

1.1 İteratif Analiz ve Toplam (Σ) Sembolü

Döngülerin işlem maliyeti, bağımlı veya bağımsız olmalarına göre toplam formülleriyle hesaplanır. Siber güvenlik açısından bir API'de darboğaz tespiti yaparken iç içe döngülerin asimptotik sınırları hayati önem taşır.

```
for i = 1 to n:
  for j = i+1 to n:
    // Sabit zamanlı (c) işlem
    ProcessData()
```

Yukarıdaki sözde kodun matematiksel analizi:

$$\begin{aligned}\sum_{(i=1 \text{ to } n)} \sum_{(j=i+1 \text{ to } n)} 1 &= \sum_{(i=1 \text{ to } n)} (n - i) \\ &= n(n) - [n(n+1)/2] = \frac{1}{2}n^2 - \frac{1}{2}n\end{aligned}$$

En yüksek dereceli terim alındığında (katsayılar ihmal edilir), bu algoritmanın karmaşıklığı $\Theta(n^2)$ olarak bulunur.

1.2 Rekürsif Analiz ve Master Teoremi

Parçala ve Fethet (Divide & Conquer) algoritmalarının zaman karmaşıklığı **Master Teoremi** ile hesaplanır. Karakteristik denklem formatı:

$$T(n) = aT(n/b) + f(n)$$

- a : Problemin bölündüğü alt problem sayısı ($a \geq 1$).
- b : Problemin ne oranda küçüldüğü ($b > 1$).
- $f(n)$: Bölme ve birleştirme işlemlerinin toplam maliyeti.

Örnek: Merge Sort Analizi

Merge Sort, diziyi daima ikiye böler ($a=2$, $b=2$) ve birleştirme işlemi için doğrusal zaman harcar ($f(n) = \Theta(n)$).

Denklem: $T(n) = 2T(n/2) + \Theta(n)$

Karşılaştırma: $n^{\log_b(a)} = n^{\log_2(2)} = n^1 \cdot f(n)$ ile eşit olduğundan Master Teoremi Durum 2 geçerlidir: $O(n \log n)$.

2. Dönüştür ve Yönet (Transform & Conquer)

Problemi doğrudan (brute-force) çözmek yerine, daha kolay işlenebilir bir forma (veri yapısı, farklı bir problem, gösterim değişikliği) dönüştürme prensibidir.

2.1 Kayıp Sayıyı Bulma Problemi (Gösterim Değişikliği)

Problem: 1 ile n arasındaki tam sayıları içeren bir dizide 1 adet sayı eksiktir. Eksik sayıyı bulmak.

Klasik toplam formülü ($n(n+1)/2$) kullanmak, büyük veri setlerinde bellek sınırlarını aşarak *Integer Overflow* zafiyetine yol açar. Güvenli ve optimize çözüm işlemci seviyesinde çalışan **XOR (Dışlayan Veya)** operatörünü kullanmaktır.

```
int findMissingNumber(int arr[], int size, int n) {
    int xor_full = 0;
    int xor_arr = 0;

    // 1'den n'e kadar olan tüm sayıları XOR'la
    for (int i = 1; i <= n; i++) {
        xor_full ^= i;
    }

    // Dizideki mevcut sayıları XOR'la
    for (int i = 0; i < size; i++) {
        xor_arr ^= arr[i];
    }

    // Aynı olan sayılar ( $A \wedge A = 0$ ) kuralıyla birbirini yok eder.
    return xor_full ^ xor_arr;
}
```

Performans: Zaman Karmaşıklığı: $O(n)$. Alan Karmaşıklığı: $O(1)$.

3. Dinamik Programlama (0-1 Sırt Çantası - Knapsack)

Örtüşen alt problemlere (overlapping subproblems) sahip durumlar için aynı hesaplamaları tekrar yapmak yerine, sonuçları bir bellekte saklayarak (Memoization/Tabulation) üstel zaman karmaşıklığını polinomsal düzeye çeken yaklaşımdır.

3.1 Problem Tanımı ve Durum Geçiş

Kapasitesi W olan bir çantaya, değerleri (v_i) ve ağırlıkları (w_i) olan eşyalar, toplam değer maksimize edilecek şekilde konulmalıdır. Eşyalar parçalanamaz (0 veya 1 durumu).

Durum Geçiş Denklemi (Bellman Equation)

$$V[i, w] = \max(V[i-1, w], v_i + V[i-1, w - w_i])$$

Her adımda eşyayı çantaya alıp almama durumu karşılaştırılır ve yerel maksimumlar tabloya yazılır. Algoritmanın maliyeti kapasite ve eşya sayısına bağlı olarak $O(n \cdot W)$ seviyesindedir.

4. Karmaşıklık Sınıfları ve Algoritma - Olasılık İlişkisi

Hesaplama teorisi, bilgisayarların pratikte çözebileceği ve çözemeyeceği problemlerin sınırlarını çizer. Sistem güvenliği ve kriptografi, temelini bu limitlerden alır.

Sınıf	Mühendislik Tanımı	Siber Güvenlik Örneği
P (Polynomial)	Polinomsal zamanda $O(n^k)$ makul bir sürede <i>çözülebilen</i> problemlerdir.	Simetrik anahtar (AES) şifreleme ve şifre çözme işlemleri.
NP (Nondeterministic Pol.)	Polinomsal zamanda çözümü <i>doğrulanabilen</i> ancak bulunamayan problemlerdir.	Açık anahtarlı şifrelemelerde büyük asal sayıları çarpanlarına ayırma (RSA).
NP-Tam (NP-Complete)	NP sınıfının en zor problemleridir. Bir NP-Tam problemi P'ye indirgenirse (Reduction), tüm NP problemleri polinom zamanda çözülebilir.	Gezgin Satıcı Problemi (TSP), 3-Coloring, Kriptografik hash çarpışmaları.

Olasılık İlişkisi: Kesin çözüm bulunması imkansız veya çok maliyetli olan NP-Tam problemlerinde (örneğin ağ yönlendirme ve sezgisel sızma testleri), *Monte Carlo* veya *Las Vegas* gibi olasılıksal/rastgele (Randomized) algoritmalar kullanılarak, belirli bir doğruluk payıyla en uygun sonuca polinom zamanda ulaşılır.